



# LangGraph Teaching Guide

A hands-on reference for students — use this alongside `mental_wellness_graph.py` to understand how to build AI workflows as graphs.

# What is LangGraph?

LangGraph lets you build **AI workflows as graphs**. Instead of one giant function, you break logic into small steps (nodes) connected by arrows (edges). Think of it like a flowchart that actually runs.

## Regular Code

```
do_step_1()
do_step_2()
do_step_3()
```

## LangGraph

```
[Node A] -> [Node B] -> [Node C]
      \      /
      +->[Node D]-+
```

### Parallel Steps

Nodes can run  
simultaneously for speed

### Conditional Routing

Route to different paths  
based on state

### Auto State

State managed  
automatically — no globals

### Easy Debug

Visualize and inspect the  
graph flow

# Core Concept #1: State

State is a container holding **all data flowing through your graph**. Every node can read from it and write to it. Define it as a Pydantic model for type safety.

```
from pydantic import BaseModel

class WellnessState(BaseModel):
    user_feeling: str = ""
    breathing_suggestion: str = ""
    final_suggestion: str = ""
```

## → Define as a Pydantic model

Gives you type safety and validation

## → Every field needs a default value

Nodes return a dict with ONLY the fields they update

## → LangGraph merges updates automatically

In our project: WellnessState has 8 fields; each node writes to its own field(s)



# Core Concept #2: Nodes

A node is a Python function that does **ONE job**. It takes state in, returns updates out.

```
def understand_mood(state: WellnessState) -> dict:  
    response = llm.invoke(f"The user feels: {state.user_feeling}")  
    return {"messages": ["Mood understood"]}
```

## Input

Takes the **full state** as input

## Output

Returns a dict with **only changed fields**

## Responsibility

Does **one thing well** — no side effects

📋 In our project: 7 nodes — understand\_mood, 3 suggest nodes, pick\_best\_practice, quick\_practice, deep\_practice.

# Core Concept #3: Edges

Edges are arrows connecting nodes. They tell LangGraph: *"after this node, run that node."*

```
graph.add_edge(START, "understand_mood")    # entry point
graph.add_edge("understand_mood", "suggest_breathing") # A -> B
```



## START

Special constant marking where the graph begins



## Directed

Edges are one-directional — flow goes one way only



## END

Special constant marking where the graph finishes

# Core Concept #4: Parallel Execution (Fan-Out)

Adding multiple edges **from the same node** causes LangGraph to run them all simultaneously.

```
graph.add_edge("understand_mood", "suggest_breathing")
graph.add_edge("understand_mood", "suggest_mindfulness")
graph.add_edge("understand_mood", "suggest_movement")
```



**Sequential**

$3 \times 2 \text{ sec} = 6 \text{ seconds}$



**Parallel**

All at once = **2 seconds**



## Important Rule

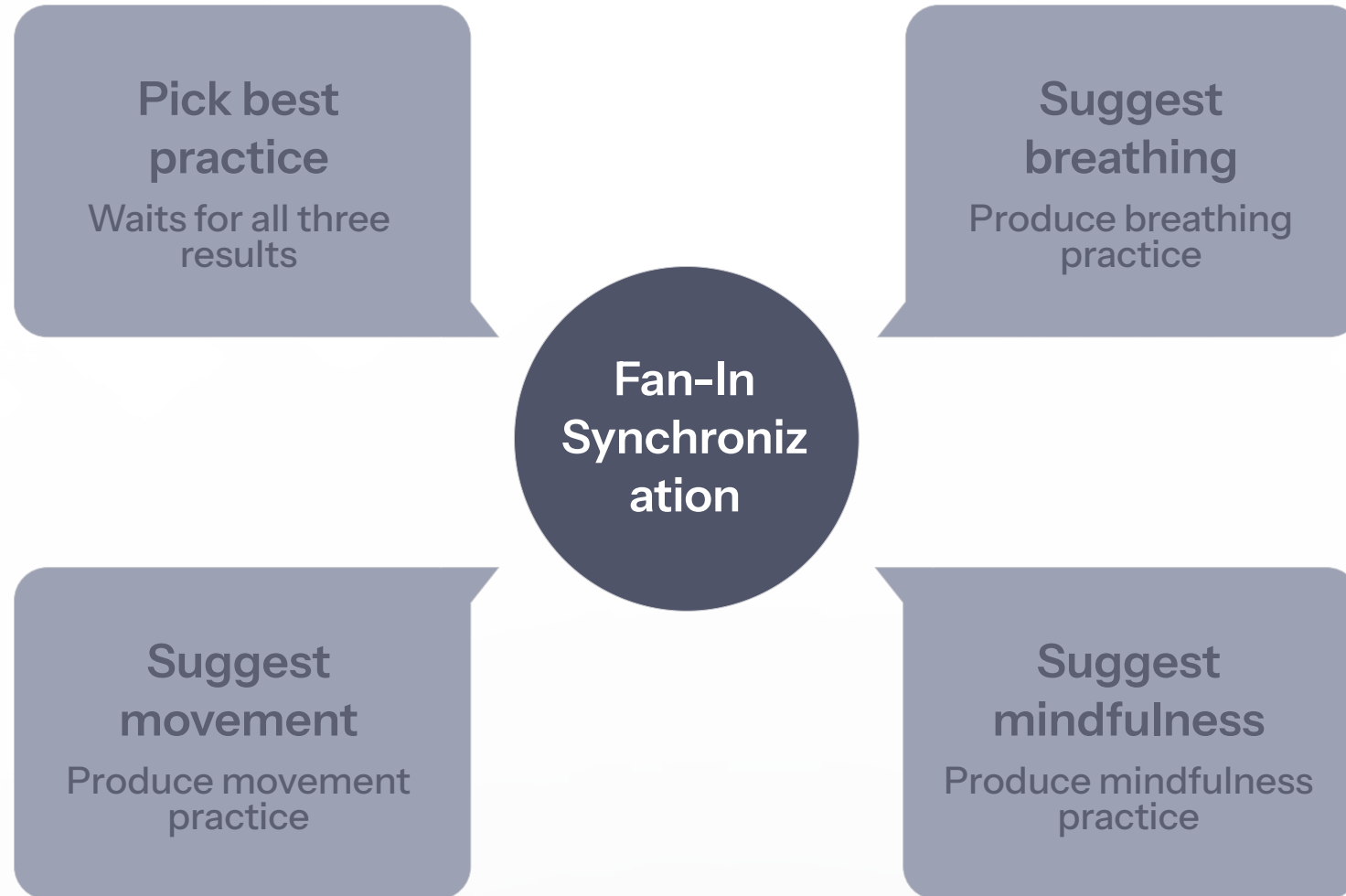
Parallel nodes must write to **different state fields**, otherwise they overwrite each other.

Exception: lists declared with `Annotated[list, operator.add]` will **append** instead of overwrite.

## Core Concept #5: Fan-In

When multiple edges point **to the same node**, LangGraph waits for **all of them** to finish before running that node. State updates are merged automatically.

```
graph.add_edge("suggest_breathing", "pick_best_practice")
graph.add_edge("suggest_mindfulness", "pick_best_practice")
graph.add_edge("suggest_movement", "pick_best_practice")
```



`pick_best_practice` only runs after all 3 suggest nodes are done — this is the Fan-In pattern, the natural complement to Fan-Out.

# Core Concept #6: Conditional Edges

Conditional edges let you **choose which node runs next** based on the current state — enabling dynamic routing.

```
def route_after_decision(state: WellnessState) -> str:
    if state.needs_deep_session:
        return "deep"
    else:
        return "quick"

graph.add_conditional_edges(
    "pick_best_practice", # from this node
    route_after_decision, # call this function
    {"quick": "quick_practice", "deep": "deep_practice"}
)
```

01

## Call routing function

LangGraph calls `route_after_decision` with current state

02

## Return a string key

Function returns `"quick"` or `"deep"`

03

## Look up in mapping dict

LangGraph maps the key to the corresponding node and routes there



# Putting It All Together

Follow these 7 steps to build any LangGraph project from scratch.



## Define State

What data flows through the graph?



## Initialize LLM

Set up your language model



## Write Nodes & Routing Functions

One job per node; routing functions for conditional paths



## Create Graph, Add Nodes & Edges

Include parallel and conditional edges



## Compile & Invoke

```
app = graph.compile() then app.invoke({...})
```

# Bonus Concept: Message Accumulation

When parallel nodes all write to the same list, use `operator.add` to **concatenate** instead of overwrite.

```
from typing import Annotated
import operator

class WellnessState(BaseModel):
    messages: Annotated[list, operator.add] = []
```

## Without `operator.add`

Parallel nodes **overwrite** each other's messages — only the last one survives.

## With `operator.add`

Node A: ["A done"]

Node B: ["B done"]

Node C: ["C done"]

Final: ["A done", "B done", "C done"]

# Common Graph Patterns

## Pattern 1: Linear Chain

START → A → B → C → END

Simple. Each node runs after the previous one.

## Pattern 2: Fan-Out / Fan-In

START → A → B1 \ B2 → C → END / B3

A triggers B1, B2, B3 in parallel. C waits for all three.

## Pattern 3: Conditional Routing

START → A → B → [decision] → C1 → END or C2 → END

Based on state, route to C1 or C2.

## Pattern 4: Our Project (all three combined!)

START → understand\_mood → suggest\_breathing \  
suggest\_mindfulness → pick\_best → [cond] → quick → END  
suggest\_movement / → deep → END

# Student Exercises

1

## Change the Domain

Replace wellness with a study planner, recipe suggester, or travel advisor. Keep the same graph structure, change the node prompts.

2

## Add a Fourth Parallel Node

Add a `suggest_journaling` node running in parallel. Update state, add the node, add edges.

3

## Add a Third Conditional Path

Add a "crisis" path for severe inputs that suggests professional help.

4

## Add User Input Loop

Study how the `__main__` block creates an interactive loop (already done in our project!).

5

## Change the LLM

Try `temperature=0.0` for factual responses vs. `temperature=1.0` for creative ones.

# Debugging Tips



## Check the Message Log

Shows which nodes ran and in what order



## Parallel Conflicts

Ensure parallel nodes write to **different** state fields



## Print the Full State

`print(result)` after `app.invoke()` shows all fields



## Graph Visualization

Open `architecture.drawio` in VS Code to see the full flow



## JSON Parse Errors

If the LLM doesn't return valid JSON, add a `try/except` fallback

# Quick Reference

| What         | Code                                                   |
|--------------|--------------------------------------------------------|
| Define state | <code>class MyState(BaseModel): ...</code>             |
| Create graph | <code>graph = StateGraph(MyState)</code>               |
| Add node     | <code>graph.add_node("name", function)</code>          |
| Add edge     | <code>graph.add_edge("from", "to")</code>              |
| Entry point  | <code>graph.add_edge(START, "first_node")</code>       |
| Exit point   | <code>graph.add_edge("last_node", END)</code>          |
| Parallel     | Multiple <code>add_edge</code> from same source        |
| Conditional  | <code>graph.add_conditional_edges(src, fn, map)</code> |
| Compile      | <code>app = graph.compile()</code>                     |
| Run          | <code>result = app.invoke({...})</code>                |
| Append lists | <code>Annotated[list, operator.add]</code>             |